Homework 4: Physics and WebXR Interactions

Contents

1 Physics with Cannon-ES.js 1

2 WebXR Interactions 5

3 Homework Assignment 11

4 Resources 12

This homework is for students in the graduate-level course 16:332:571 VR Technology. It combines material typically covered in two undergraduate lab sessions: **Physics Simulation** and **WebXR Interactions**.

In this homework, you will learn how to synchronize a visual 3D world with a physics engine (Cannon-ES.js) and how to implement complex interactions (dragging and spawning) in VR using Raycasting and WebXR input events.

Sections 1 and 2 are guided tutorials. Homework requirements are detailed in Section 3.

1 Physics with Cannon-ES.js

In this section, we will explore **physics simulation** using **Cannon-ES.js**, a JavaScript physics engine that works seamlessly with Three.js.

You will learn how physics engines maintain **two separate worlds** (visual and physics) that must be synchronized. You will also learn how to use the **cannon-es-debugger** to visualize physics bodies and how to debug common physics issues by identifying mismatches between visual meshes and physics bodies.

Physics engines maintain **two separate worlds**: the visual world (Three.js) and the physics world (Cannon-ES). These worlds must be precisely synchronized every frame.

- **Three.js scene** is the visual world that you see.
- **Cannon-ES world** is the physics world where calculations happen.
- **Sync loop** copies physics positions to visual meshes each frame so that they remain synchronized.

Every object needs both a visual mesh (Three.js) and a physics body (Cannon-ES). They must have **matching positions, sizes, and rotations**.

Note: `CANNON.Box` takes half-extents as arguments, while `THREE.BoxGeometry` takes full extents as arguments. For sphere, both `CANNON.Sphere` and `THREE.SphereGeometry` take radius.

1.1 Watch the Videos

Watch the following YouTube videos to understand the basics of physics engines:

- Three.js + Cannon.js Tutorial (Part 1) (~8 min)
- Three.js + Cannon.js Tutorial (Part 2) (~10 min)

1.2 Experiment with the Reference Implementation

1. Open the `hw4` folder in VSCode.
2. Serve `hw4_reference_physics.html` using **Live Server**.
3. **Try the Controls**:
   - **S**: Drop a **S**phere
   - **B**: Drop a **B**ox
   - **C**: **C**lear Scene
   - **D**: Toggle **D**ebug Wireframes

Open `hw4_reference_physics.html` in VS Code and study the code structure. Observe how every object needs both a visual mesh (Three.js) and a physics body (Cannon-ES). They must have **matching positions, sizes, and rotations**.

1.3 Part 1: Fix the Bugs

Physics simulations break when the visual world and the physics world don't match. Your mission is to find and fix the bugs. Please use the `hw4_buggy_physics.html` file in your `hw4` folder.

1.3.1 Setup

1. Open the `hw4` folder in VSCode and serve `hw4_buggy_physics.html` using **Live Server**.
2. Press **S** to spawn spheres.
3. Press **B** to spawn boxes.
4. Press **D** to enable debug mode.

1.3.2 Observe the Issues

Something is wrong. Use the debug wireframes to identify the issues:

- Position mismatches
- Size mismatches
- Rotation mismatches
- Missing sync

1.3.3 Find the Bugs

1. Open `hw4_buggy_physics.html` in VS Code.

2. **Compare it to `hw4_reference_physics.html`** (the correct version).
3. For each bug you identify, please:
    - **Fix it directly in the code.**
    - Briefly write down what the bug was and how you resolved it in your report.

1.3.4 Verify Your Fixes

1. Fix each bug in `hw4_buggy_physics.html`.
2. Test by hard refreshing (Shift + Ctrl + R) or (Shift + Command + R on Mac) the browser.
3. Confirm the debug wireframes now match the visual meshes.
4. Rename your fixed html file to `hw4_physics_[NetID].html` when submitting.

1.4 Part 2: The Rutgers R Challenge

Now that you have fixed the bugs, it is time for the Rutgers R Challenge. We want to create a simulation where **Rutgers "R" logos fall from the sky**.

1.4.1 Setup

1. Verify `RutgersR.glb` is in your `hw4` folder.
2. Open your **fixed** `hw4_buggy_physics.html`.
3. (Optional) Open `example_gltf_loader.html` in your `hw4` folder using Live Server to see a working example of loading the model without physics.

At this point, your `hw4` folder should look like this:

```
hw4/
├── hw4_buggy_physics.html
├── RutgersR.glb
└── example_gltf_loader.html
```

1.4.2 Implement the Feature

The code for loading the Rutgers R model `RutgersR.glb` is already included in your html file (originally from `lab07_buggy.html`). It uses `GLTFLoader.load( url : string, onLoad : function, onProgress : onProgressCallback, onError : onErrorCallback )` to load the model. When loading finishes, the `onLoad` callback runs and receives a `gltf` object, which contains the loaded model as `gltf.scene`. Then callback adds `gltf.scene` to the Three.js scene, applies scaling to the model, enables shadows, and calculates the bounding box of the scaled model.

The included code should work as long as `RutgersR.glb` is in the same folder as your html file, and you are serving the page via **Live Server** (not opening as a file).

For this part, you may find it helpful to check the **Developer Console** for errors:

1. Press **F12** or **Ctrl+Shift+I** (or **Cmd+Option+I** on Mac) to open the developer console.

2. Switch to the **Console** tab at the top of the panel.
3. Look for red error messages.

In this lab, there are two layers of debugging (both involve JavaScript):

- **Application Layer**: Debugging the relationship between the visual world (Three.js) and the physics world (Cannon-ES) using the debug wireframes to check for mismatches.
- **JavaScript Layer**: Debugging the code itself, e.g., syntax errors, reference errors, logic bugs, etc., using the browser's developer tools.

Implement the `spawnR()` function so that pressing the **R** key performs the following actions:

1. Clones the loaded R model (`model.clone()`).
2. Adds it to the scene at a random X and Z position but with a fixed height (e.g., Y=6). You can take a look how this is done in `createSphere()` and `createCube()`, and reuse the same logic.
3. Creates a matching **Cannon-ES box body** (using the dimensions of the model you found) to approximate the shape.
4. Syncs them in the animation loop.

Hint: The provided html file already contains the following code to first obtain the size of the model using `THREE.Box3().setFromObject(rModel)` and then calculate the half extents needed to set the Cannon-ES box.

```
const box = new THREE.Box3().setFromObject(rModel);
const size = new THREE.Vector3();
box.getSize(size);

rHalfExtents = new CANNON.Vec3(size.x / 2, size.y / 2, size.z / 2);
```

1.4.3 Add Controls

Once you have completed the implementation of `spawnR()`, uncomment the following code in the provided html file so that pressing **R** calls `spawnR()`.

```
<!-- TODO: uncomment below once spawnR() is fully implemented -->
<!-- <span class="key">R} Rutgers R   -->
```

```
// TODO: uncomment below once spawnR() is fully implemented
// case 'r':
//     spawnR();
//     break;
```

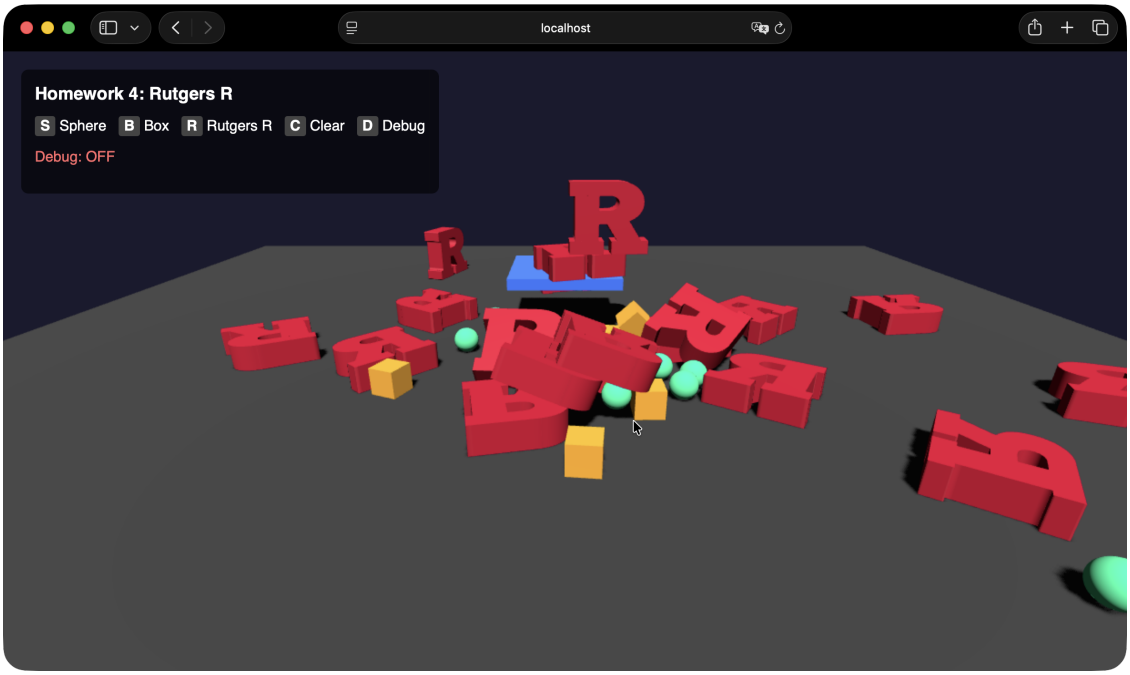


Figure 1: The Rutgers R Challenge

2 WebXR Interactions

In this section, we will explore **WebXR interactions** using **Three.js**. You will learn how to use **raycasting** to interact with 3D objects in VR, specifically, how to **drag** objects using the controller and how to **spawn** new objects into the scene.

2.1 Setup: WebXR Emulator

To complete the WebXR interaction tasks, you will need to use the Chrome Browser with the **Immersive Web Emulator** extension.

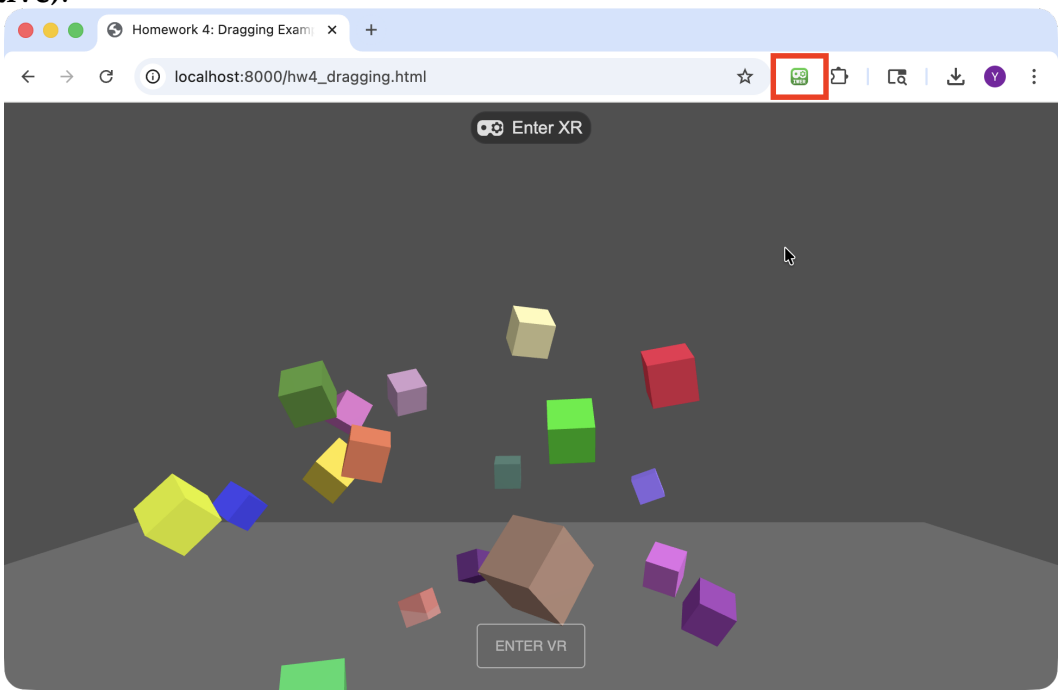
2.1.1 Install the Immersive Web Emulator

1. Download the **v2.0.0-alpha** release: iwe-v2.0.0-alpha-release.zip.
   - **Note:** Do NOT use the version from the Chrome Web Store. It is outdated.
2. Unzip the file.
3. Open Chrome Extensions (`chrome://extensions`).
4. Enable **Developer Mode** (top right toggle).
5. Click **Load Unpacked** and select the unzipped folder.

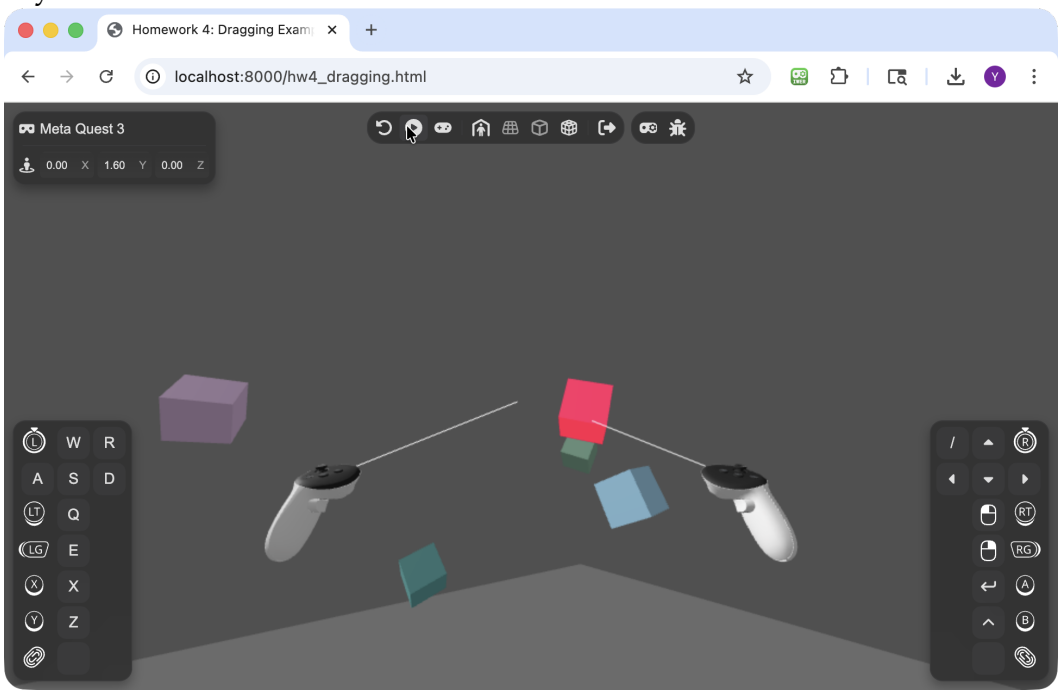
2.1.2 Enter VR Mode in the Emulator

1. Reload your page in the Chrome Browser.

2. Click the **Emulator Icon** in your browser toolbar (top right) to toggle it **Green (Active)**.



3. Click **"ENTER VR"**.
4. **Enable Movement:** At the top of the page, click the **Play Button** (▶) in the emulator overlay.



5. Now you can look around by clicking and dragging with mouse.

2.2 Part 1: Study the Example Files

Both examples use **raycasting**, the process of shooting an invisible line from the controller into the 3D scene to detect what it intersects with. Open each file in VS Code and study how they work.

2.2.1 Scene Setup

The `init()` function begins by setting up the 3D environment, including the lighting, camera, floor, and 20 interactable objects (cubes) placed at random positions and orientations. It then initializes the two VR controllers and their corresponding 3D models so they are visible in the scene.

It constructs a new Raycaster, which will be used for obtaining ray-object intersections. Check out more information regarding the Raycaster here: https://threejs.org/docs/#Raycaster

```
raycaster = new THREE.Raycaster();
```

Finally, it configures the renderer to call the `animate()` function every frame, and then enables WebXR support:

```
renderer.setAnimationLoop(animate);
renderer.xr.enabled = true;
```

The `animate()` function is the main render loop that runs every single frame. It is responsible for handling continuous per-frame logic, such as hover effects and updating the screen.

```
function animate() {
    // Reset any hover effects from the previous frame.
    cleanIntersected();

    // Perform raycasting for both controllers to check if they are currently
     ↪  intersecting with any objects
    // If they are, it highlights the objects in red
    intersectObjects(controller1);
    intersectObjects(controller2);

    // Render the updated scene
    renderer.render(scene, camera);
}
```

2.2.2 Controller Setup

The following javascript code sets up the controllers:

```
// Get the first controller (index 0)
controller1 = renderer.xr.getController(0);
// Listen for 'selectstart' (trigger pressed down)
controller1.addEventListener('selectstart', onSelectStart);
// Listen for 'selectend' (trigger released)
controller1.addEventListener('selectend', onSelectEnd);
scene.add(controller1);
```

This sets up the controllers to listen for two events: `selectstart` (**trigger** pressed down) and `selectend` (**trigger** released). Upon **trigger** press, a function `onSelectStart`, defined later in the code, is called. This function is a listener which will be called with details about the event. Similarly, upon **trigger** release, `onSelectEnd` is called.

2.2.3 Visualization of Rays

The code below visualizes the rays extended from the two controllers. Note that these rays are for visualization only. They are not used for the actual RayCasting for intersection with objects.

```
// Create a line from the origin (0,0,0) down the negative Z-axis (forward).
const lineGeometry = new THREE.BufferGeometry().setFromPoints([new
↪  THREE.Vector3(0, 0, 0), new THREE.Vector3(0, 0, - 1)]);
const line = new THREE.Line(lineGeometry);
line.name = 'line';
line.scale.z = 5; // Set the laser length to 5 meters.

// Attach a copy of the laser line to both controllers,
// so that they point to the -Z direction from the controller's perspectives.
controller1.add(line.clone());
controller2.add(line.clone());
```

2.2.4 Dragging (`hw4_dragging.html`)

When you point at an object and hold the **trigger** (`selectstart` event), the code raycasts and attaches the hit object to the controller. This allows you to pick up / grab and move objects in the scene.

```
function onSelectStart(event) {
    const controller = event.target;
```

```
    raycaster.setFromXRController(controller);
    const objectIntersections = raycaster.intersectObjects(group.children,
    ↪  false);

    if (objectIntersections.length > 0) {
        const object = objectIntersections[0].object;
        object.material.emissive.b = 1; // Highlight blue
        controller.attach(object); // Attach to the controller
        controller.userData.selected = object;
    }
}
```

When the **trigger** is released (`selectend`), the object is detached and placed back in the scene. At this point, the movement of the object is independent from the controller.

```
function onSelectEnd(event) {
    const controller = event.target;

    if ( controller.userData.selected !== undefined ) {
        const object = controller.userData.selected;
        object.material.emissive.b = 0; // Remove highlight
        group.attach(object); // Attach back to the scene group
        controller.userData.selected = undefined;
    }
}
```

2.2.5 Spawning (`hw4_spawning.html`)

Instead of grabbing existing objects, this example **creates new objects** where you point. When the **trigger** is pressed, a raycast checks if it hits the floor. If it does, a new cube is spawned at the intersection point. If not, a new cube is spawned 3 meters along the ray.

```
function onSelectStart(event) {
    const controller = event.target;

    let spawnPoint;

    raycaster.setFromXRController(controller);
    const intersects = raycaster.intersectObject(floor);

    if (intersects.length > 0) {
        // If hit the floor, spawn at intersection
        spawnPoint = intersects[0].point.clone();
```

```
        spawnPoint.y += 0.075;
    } else {
        // If did not hit the floor, then spawn 3 meters along the ray
        // spawnPoint = raycaster.ray.origin + 3 * raycaster.ray.direction
        spawnPoint = new THREE.Vector3();
        spawnPoint.copy(raycaster.ray.direction).multiplyScalar(3)
        spawnPoint.add(raycaster.ray.origin);
    }

    // Create a new object
    const geometry = new THREE.BoxGeometry(0.2, 0.2, 0.2);
    const material = new THREE.MeshStandardMaterial({
        color: Math.random() * 0xffffff,
        roughness: 0.7,
        metalness: 0.0
    });
    const newObject = new THREE.Mesh(geometry, material);
    newObject.position.copy(spawnPoint);

    scene.add(newObject);
}
```

2.3 Part 2: Combine Both Interactions

Your goal is to build a VR scene that supports **both** dragging and spawning using **different controller buttons**.

First, duplicate your `hw4_dragging.html` file and rename the new copy to `hw4_combined_[NetID].html`. You will use this as your starter file.

Based on the concepts you studied in the examples, implement the following:

Controller Button	WebXR Event	Action
Trigger	`selectstart` / `selectend`	**Grab and drag** an existing object (already implemented in your base file)
Grip	`squeezestart`	**Spawn** a new object where you are pointing

2.3.1 Hints

1. To listen for the **grip** button, you need to add an event listener for `squeezestart`:

```
controller.addEventListener('squeezestart', onSqueezeStart);
```

Do so for both controllers.

2. You need to implement the `onSqueezeStart` function. Since spawning happens instantaneously on clicking the **grip** button, you don't need to worry about implementing a `squeezeend` event or function. The logic should be very similar to `onSelectStart` from `hw4_spawning.html`.
3. If you add new objects to the `group` (instead of the `scene`), they will automatically become draggable by the **trigger** button because the dragging logic checks for intersections with `group.children`:

```
group.add(newObject);  // Now it becomes part of the draggable group
```

3 Homework Assignment

3.1 Task 1: Physics Synchronization and Debugging

1. In `hw4_buggy_physics.html`, identify and fix **at least 4 physics-visual mismatches**.
2. Fix each bug in the code.
3. In your report, list each bug found and how you fixed it.

3.2 Task 2: The Rutgers R Challenge

Once you have fixed the bugs in `hw4_buggy_physics.html`, duplicate it and rename the new copy to `hw4_physics_[NetID].html`. Complete the implementation of the `spawnR()` function in your `hw4_physics_[NetID].html` so that the **R** key spawns falling Rutgers R models with correct physics.

3.3 Task 3: Multi-Interaction VR Scene

Create `hw4_combined_[NetID].html` by combining the dragging and spawning logic as described in Section 2.

- **Trigger** should drag objects.
- **Grip** (Squeeze) should spawn new objects.
- Spawned objects must be draggable.

3.4 Submission

Submit four files for your homework:

1. **A report** named `hw4_[NetID].pdf` containing:
   - **Task 1**: Notes on fixed bugs.
   - **Task 2**: Screenshot of falling Rutgers R models, spheres, and boxes with debug wireframes ON (green wireframes matching the 3D objects).

- **Task 3**: Screenshot of your interaction in VR showing both original and spawned objects.

2. **A `.zip` file** named `hw4_[NetID].zip` (e.g., `hw4_ab123.zip`) containing all your modified html files:
   - `hw4_physics_[NetID].html` (Your final code with all bugs fixed and the "Rutgers R" challenge implemented).
   - `hw4_combined_[NetID].html` (Your final code with both interactions working).
3. **A `.mp4` file** of a short **screen recording** (~10s) showing:
   - Falling Rutgers R models, spheres, and boxes with working physics.
4. **A `.mp4` file** of a short **screen recording** (~10s) showing:
   - Spawning new objects by pressing the **grip** button.
   - Selecting and dragging objects by squeezing the **trigger** button.

3.5 Grading

Component	Points	Requirement
Task 1: Physics Debugging	15	Identify and fix bugs in `hw4_buggy_physics.html`. List the bugs and fixes in your report. Include the screenshots as required.
Task 2: The Rutgers R Challenge	15	Complete the implementation of the `spawnR()` function in your `hw4_physics_[NetID].html` so that the **R** key spawns falling Rutgers R models with correct physics. Include the screenshots as required. Submit a screen recording showing the working physics.
Task 3: Multi-Interaction VR Scene	20	Create `hw4_combined_[NetID].html` by combining the dragging and spawning logic as described in Section 2. Include the screenshots as required. Submit a screen recording showing the working interaction.
Total	**50**	

4 Resources

- Debug Javascript
- Cannon-ES Documentation
- Cannon-ES Debugger
- Three.js GLTFLoader Docs
- Three.js GLTFLoader Example
- Three.js + Cannon.js Tutorial
- Three.js WebXR Examples
- Three.js WebXR Documentation
- MDN WebXR Device API
- Immersive Web Emulator

- webxr_xr_dragging.html
- Three.js Raycaster
- Meta's WebXR First Steps Tutorial